

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	CH.Bhavya Sri	Reg. No.:	192411249
Section / Batch:	Slot c	Date:	25.08.2026

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – Pseudocode Writing Task

Write a pseudocode using Backtracking to count the total number of valid ways to place N queens on an $N \times N$ chessboard. Clearly define inputs (board size N) and output (total number of valid solutions). Design a complete algorithm that explores all possible queen placements row by row. Use loops and recursive branching effectively to generate and validate configurations. Apply pruning whenever a queen placement violates row, column, or diagonal safety rules. Present the pseudocode in a clear and structured format. Ensure the solution handles all configurations and counts only valid arrangements without omission.

Code of Conduct

I CH.Bhavya Sri/192411249_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.Bhavya Sri_____

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm Design	30%				
Use of Control Structures	20%				
Pseudocode Structure	10%				
Completeness	20%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

CHENNABOINA BHAVYA SRI
192411249RC

CO5 AT2-Pseudocode Writing Task (Algorithm Design)

← Back

Language: Python C C++ Java

QUESTIONS

Q1
N-Queens Problem for Counting All Solutions
100 marks

0/1 answered

Q1 N-Queens Problem for Counting All Solutions100 marks

Write a pseudocode using Backtracking to count the total number of valid ways to place N queens on an N×N chessboard. Clearly define inputs (board size N) and output (total number of valid solutions). Design a complete algorithm that explores all possible queen placements row by row. Use loops and recursive branching effectively to generate and validate configurations. Apply pruning whenever a queen placement violates row, column, or diagonal safety rules. Present the pseudocode in a clear and structured format. Ensure the solution handles all configurations and counts only valid arrangements without omission.

Your Code

```
1 def is_safe(board, row, col):
2     for i in range(row):
3         if board[i] == col:
4             return False
5         if abs(board[i] - col) == abs(i - row):
6             return False
7     return True
8
9 def solve_n_queens(board, row, n):
10     global count
11     if row == n:
```

Input

4

Output

2

EXPLANATION:

The **N-Queens problem** means placing **N queens on an $N \times N$ chessboard** so that **no two queens attack each other**.

1. Input and Output

- **Input:** N — size of the chessboard.
- **Output:** Total number of valid arrangements.

For example:

Input: 4

Output: 2

So, there are **2 possible solutions** for a 4×4 board.

2. Main idea

We place queens **one row at a time**.

For every row, we try placing the queen in every column.

Before placing, we check whether that position is safe.

3. Safety conditions

A queen cannot be placed if another queen is already in:

- **Same column**
- **Same diagonal**
- Same row is automatically avoided because we place only **one queen per row**.

4. Backtracking

The process is:

Place Queen



Check if safe



Yes → Move to next row



No → Try next column



If no column works → Go back to previous row

This going back and trying another position is called **backtracking**.

5. Pruning

If a position is unsafe, we **immediately reject it** and don't continue that branch.

For example:

Row 1 → Column 1

Row 2 → Column 1 ✗

We don't continue with this arrangement because two queens are in the same column.

6. Counting solutions

Whenever we successfully place queens in **all N rows**, one complete valid solution is found.

So:

if row == N:

 count = count + 1

Finally, the count gives the **total number of valid solutions**.